Entrega final: Una red social

Como entrega final del módulo vas a construir una red social tipo Twitter. Para ello, te doy el programa principal (`main.py`) y a partir de él tendrás que inferir, con ayuda del enunciado, qué clases es necesario implementar para hacerlo funcionar.

En `punto_de_partida/main.py` tienes una pequeña red social tipo Twitter ya escrita (usuarios que se siguen, tweets, respuestas, retweets, hashtags y tendencias), pero le falta todo el paquete `red_social/` del que hace `import`, que contiene las clases para hacer funcionar el programa. La tarea consiste en escribir el paquete con todas las clases necesarias y hacer que `main.py` produzca exactamente la salida que se muestra más abajo en el enunciado. Además, tendrás que añadir los tests que se piden.

> **Requisitos previos y entrega**
>
> - Crea el proyecto con `uv init red_social --python 3.12` y añade las dependencias de desarrollo: `uv add --dev ruff pytest pytest-mock`.
> - Copia dentro `main.py` y la carpeta `datos/` sin modificarlos.
> - Estructura obligatoria:
>
> ```
> red_social/            # raíz del proyecto
> ├── pyproject.toml
> ├── main.py            # el que te damos, sin tocar
> ├── datos/usuarios.json
> ├── red_social/        # el paquete que tienes que escribir
> │   ├── usuario.py
> │   ├── publicacion.py
> │   └── red.py
> └── tests/
>     ├── conftest.py
>     ├── test_usuario.py
>     ├── test_publicacion.py
>     └── test_red.py
> ```

PontIA

Parte 1: Las clases

Lo que sigue son los requisitos mínimos de cada una, sin código. Puedes añadir cosas extra si las necesitas, pero lo que está aquí tiene que existir con estos nombres.

`red_social/usuario.py` — clase `Usuario`

Constructor: `Usuario(nombre, alias)`.

Atributos:

- `nombre`.
- `alias`: el nombre de usuario, que empieza siempre por `@`. Si al constructor le llega un alias sin `@`, se lo añade.
- `seguidos`: lista de usuarios a los que sigue, vacía al crearlo. Cada usuario tiene la suya.

Métodos:

- `seguir(otro)`: añade a `otro` a `seguidos`. Lanza `ValueError` si `otro` es él mismo o si ya lo sigue.
- `sigue_a(otro)` → `True` / `False`.
- `a_dict()` → `{"nombre": "Ana", "alias": "@ana"}`.

Properties:

- `numero_seguidos`: cuántos usuarios sigue.

Constructor alternativo

- `desde_dict(datos)`: construye un usuario a partir de un diccionario como el que devuelve `a_dict()`.

Métodos especiales:

- `__str__` → `Ana (@ana)`.
- `__eq__`: dos usuarios son iguales si tienen el mismo alias.

`red_social/publicacion.py` — clases `Publicacion`, `Tweet`, `Respuesta` y `Retweet`

Constante de módulo: `LIMITE_CARACTERES = 280`.

`Publicacion` (clase base, abstracta)

Hereda de `ABC` y no se puede instanciar directamente. Contiene todo lo que tienen en común los tres tipos de publicación.

Constructor: `Publicacion(autor, texto)`. Valida el texto: si está vacío o solo tiene espacios, o si supera `LIMITE_CARACTERES`, lanza `ValueError`. 280 caracteres exactos es válido.

Atributos: `autor` (un `Usuario`), `texto` y `me_gusta` (empieza en 0).

Métodos:

- `dar_me_gusta()`: suma uno a `me_gusta`.

Properties:

- `hashtags`: los hashtags del propio texto (aplica `extraer_hashtags` a `self.texto`).

Método estático

PontIA

- `extraer_hashtags(texto)`: es una regla pura, solo depende de su parámetro. Devuelve la lista de hashtags del texto: una palabra que empieza por `#`, en minúscula, sin la puntuación final (`,.;:!?¡¿`), sin repetidos y en orden de aparición. Un `#` solo no es un hashtag.

Métodos especiales:

- `__str__`: abstracto (`@abstractmethod`). Cada hija decide cómo se muestra.

`Tweet(Publicacion)`

Constructor: `Tweet(autor, texto)` (el de la base sirve tal cual).

Métodos especiales: `__str__` → `@ana: Hola a todos #python #pytest`.

`Respuesta(Publicacion)`

Constructor: `Respuesta(autor, texto, original)`, donde `original` es la publicación a la que responde.

Atributos: los de la base más `original` (composición: una respuesta tiene una publicación).

Métodos especiales: `__str__` → `@luis ↩ @ana: ¡Bienvenida! #Python` (el segundo alias es el del autor de la original).

`Retweet(Publicacion)`

Constructor: `Retweet(autor, original)`. No recibe texto: **comparte el de la original** (y por tanto también sus hashtags, que cuentan en las tendencias).

Atributos: los de la base más `original`.

Métodos especiales: `__str__` → `@marta 🔁 @ana: Hola a todos #python #pytest`. Pista: lo que va después de 🔁 es exactamente `str(original)`; no lo reconstruyas a mano.

En las hijas que definan `__init__`, este tiene que llamar al de la base con `super().__init__(...)`: la validación y los atributos comunes viven en un solo sitio.

`red_social/red.py` — clase `RedSocial`

La red tiene usuarios y publicaciones (composición) y coordina las operaciones entre ellos. No hereda de nada.

Constructor: `RedSocial()`, sin parámetros.

Atributos:

- `usuarios`: diccionario alias → `Usuario`.

PontIA

- `publicaciones`: lista, en orden de publicación.

Métodos:

- `anadir(usuario)`: guarda el usuario y lo devuelve. Lanza `ValueError` si el alias ya existe.
- `registrar(nombre, alias)`: crea el `Usuario` y lo añade con `anadir`. Devuelve el usuario.
- `publicar(publicacion)`: la guarda y la devuelve (así `main.py` puede hacer `hola = red.publicar(Tweet(...))`).
- `timeline(usuario)`: lista con las publicaciones de los usuarios a los que sigue (no las suyas), la más reciente primero.
- `tendencias(n=3)`: los `n` hashtags más usados en toda la red como lista de tuplas `(hashtag, veces)`, de mayor a menor. `collections.Counter` y su método `most_common` te ahorran casi todo el trabajo.
- `mostrar_timeline(alias)`: imprime la cabecera `Timeline de Marta (@marta):` (precedida de una línea en blanco) y una línea por publicación con el formato que ves en la salida.

Constructor alternativo

- `desde_json(ruta)`: abre el fichero con `open(ruta, encoding="utf-8")`, crea los usuarios con `Usuario.desde_dict` y aplica los seguimientos. El formato es el de `datos/usuarios.json`: cada par de `"seguimientos"` es `[quien_sigue, a_quien]`.

Métodos especiales:

- `__len__`: número de usuarios.
- `__iter__`: recorre los usuarios ordenados por alias.
- `__contains__(alias)`: `"@ana" in red`.
- `__getitem__(alias)`: `red["@ana"]`. Si el alias no existe lanza `KeyError`, igual que haría un diccionario.

PontIA

Parte 2: La salida esperada

Cuando el paquete esté completo, `uv run main.py` tiene que imprimir exactamente esto:

```
Usuarios registrados: 3
  Ana (@ana) sigue a 0
  Luis (@luis) sigue a 1
  Marta (@marta) sigue a 2

Nuevo usuario: Pablo (@pablo)
¿Está @pablo en la red? True
¿Sigue @pablo a @ana? True
¿Sigue @ana a @pablo? False

Publicaciones:
  @ana: Hola a todos #python #pytest  ♥ 2
  @luis ↩ @ana: ¡Bienvenida! #Python  ♥ 0
  @marta 🔁 @ana: Hola a todos #python #pytest  ♥ 0
  @pablo: Mi primer tweet #hola  ♥ 0

Hashtags del tweet de Ana (@ana): ['#python', '#pytest']

Timeline de Marta (@marta):
  @luis ↩ @ana: ¡Bienvenida! #Python  ♥ 0
  @ana: Hola a todos #python #pytest  ♥ 2

Tendencias:
  #python (3)
  #pytest (2)
```

PORTIA

Fíjate en los detalles que se pueden deducir de aquí: el retweet de Marta hace que `#python` cuente 3 y `#pytest` 2; el tweet de Pablo no aparece en el timeline de Marta porque Marta no lo sigue; los usuarios salen ordenados por alias.

Parte 3: Los tests

Configura pytest añadiendo al final de `pyproject.toml`:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

Fixtures obligatorias (`tests/conftest.py`)

- `ana` → `Usuario("Ana", "@ana")`
- `luis` → `Usuario("Luis", "@luis")`
- `tweet` → `Tweet(ana, "Hola a todos #python #pytest")` (pide la fixture `ana`)
- `red` → una `RedSocial()` en la que se registran tres usuarios con `red.registrar(...)`: Ana (`@ana`), Luis (`@luis`) y Marta (`@marta`), y donde Luis sigue a Ana y nadie más sigue a nadie. Esta fixture no usa las fixtures `ana` y `luis`: crea sus propios usuarios dentro de la red (los tests que la usan los recuperan con `red["@ana"]`, etc.).

Los tests

Escribe estos tests, con la herramienta de pytest que se indica y que comprueben lo que dice la tabla.

Fichero	Test	Qué comprueba	Herramienta
`test_usuario.py`	`test_seguir_anade_al_otro_usuario`	tras `ana.seguir(luis)`: `sigue_a` es `True`, `numero_seguidos` es 1 y Luis **no** sigue a Ana	fixtures `ana` y `luis`
	`test_seguirse_a_si_mismo_lanza_error`	`ana.seguir(ana)` lanza `ValueError`	`pytest.raises`
`test_publicacion.py`	`test_texto_invalido_lanza_error`	`""`, `" "` y 281 caracteres lanzan `ValueError`	`parametrize` + `pytest.raises`
	`test_extraer_hashtags`	`Publicacion.extraer_hashtags(texto)` devuelve la lista esperada en 4 casos: (1) un texto sin hashtags → `[]`; (2) `"Hola #Python #PYTEST"` → `["#python", "#pytest"]` (pasa a minúscula); (3) `"#python y otra vez #python"` → `["#python"]` (sin repetidos); (4) `"Me gusta #python, y mucho #pytest!"` → `["#python", "#pytest"]` (quita la puntuación final)	`parametrize` con pares `(texto, esperado)`
	`test_retweet_comparte_el_texto_del_original`	`original is tweet`, mismo `texto`, mismos `hashtags`, `autor` es quien retuitea	fixtures `tweet` y `luis`
`test_red.py`	`test_timeline_solo_tiene_publicaciones_de_los_seguidos`	el timeline de Luis tiene solo los tweets de Ana, el más reciente primero; el de Ana está vacío	fixture `red`
	`test_desde_json_construye_usuarios_y_seguimientos`	sin tocar el disco: `open` se llamó **una vez** con la ruta y `encoding="utf-8"`, hay 2 usuarios y el seguimiento se aplicó en la dirección correcta	`mocker.mock_open`

Pontia

Con todo en su sitio:

```
uv run pytest -v
```

debería terminar con 12 casos en verde.